

Design note: dfactor

Status: SIGNED (Mico, 2026-08-01) with D2 amended: identification is imposed through a general CONSTRAINTS subsystem (Stata's constraint command), not a bespoke identify() option — general state-space models need constraints even more often than dfactor does. D1 approved as Stata-default; D3 approved (dsyev enters the wasm shim). Parent: DE-SIGN_SSPACE.md (the engine, Decisions 1-7, 9 all inherited unchanged). Governing ruling (Mico, 2026-08-01): same-named commands strive for functional compatibility with Stata — so dfactor accommodates multiple factors from the start, even though one factor is the every-day case.

1. Model and command surface

Stata's model:

$$\begin{aligned} y_t &= \text{Lambda } f_t + [B \ x_t] + u_t \\ f_t &= A_1 f_{t-1} + \dots + A_p f_{t-p} + v_t, & v_t &\sim N(0, I_k) \\ u_t &\sim N(0, \text{diag}(s2_1 \dots s2_n)) & & \text{(this release)} \end{aligned}$$

Syntax subset, matching Stata's shape:

```
dfactor (y1 y2 ... [= exoglist] [, noconstant]) (f1 [f2 ...] = , ar(#/#))
```

- k factors = number of names in the factor equation; each extra name is one more factor. Factor names are labels for output and for smoothed-factor extraction; they create no variables by themselves.
- ar(1/p) gives every factor AR(p) dynamics with a FULL A_j matrix (factors interact), Stata's default. ar(1) means $p=1$.
- Observation equation: constants included unless noconstant (Stata's default); exog regressors supported with common coefficients per equation, as in Stata.
- Options: smfactor(stub) writes smoothed factors as stub1..stubk (tea extension, mirroring ucm's smstate; Stata does this via predict, which we stage).

DECISION D1 (recommended): the surface above; k limited to 4 and p to 4 in this release ($m = n_{\text{idio-free states}} = k \cdot p$; with $n \leq 12$ observables everything stays tiny).

2. Normalization and identification

Stata's normalization is $\text{Var}(v_t) = I_k$: factor scale lives in the loadings. Adopted unchanged.

For $k \geq 2$ the model is identified only up to orthogonal rotation of the factor space. Stata runs the unrestricted model anyway; users impose identification with constraint commands. tea has no constraint language, so:

- DEFAULT: unrestricted loadings — Stata's behavior, per the compatibility ruling. The log likelihood, fitted values, and the factor SPACE are identified and reproducible; the particular rotation the optimizer lands on is not. When $k \geq 2$ and no identification option is given, tea prints one note line saying exactly that.
- Identification is imposed the Stata way: constraint definitions plus a constraints() option on the estimator (see §7). The standard lower-triangular pattern is spelled, exactly as in Stata,

```
constraint 1 [y2]f2 = 0          // for k=2, or the k(k-1)/2
dfactor (y1 y2 y3 = ) (f1 f2 = , ar(1)), constraints(1)
```

and constrained $k \geq 2$ forms are the only ones that regression goldens and the Stata check script lock.

- Sign convention, all k: if the first nonzero loading of a factor is negative at the optimum, that factor's sign is flipped (loadings and smoothed factor negated; AR coefficients are sign-invariant for the flip applied jointly). A deterministic representative of an equivalence class, not a different model.

DECISION D2 (recommended): as above. The rotation manifold is the structural cousin of the airline flat ridge (v1.6.38): the byte-identity promise attaches to identified quantities, and the goldens only ever lock identified quantities.

3. Parameterization and estimation

theta = [mu (n, unless noconstant) | B (exog) | Lambda (free entries, row-major over observables) | vec of A_1..A_p | log-sigma of the n idiosyncratic variances]. No transform on Lambda or A: Stata leaves factor dynamics unconstrained (nonstationary A during a likelihood evaluation returns +inf as in the sspace ruling — Lyapunov failure is an evaluation error, not a crash). Optimizer, starts, convergence, OIM standard errors: exactly the ucm/arima pattern (BFGS2, central differences, deterministic multistart, Hessian in theta-space — everything here is already interior/unconstrained).

Starting values (deterministic): principal components. Lambda_0 = first k PC loadings of the (standardized-in-passing) observables, scaled back; factor AR from OLS of PC scores on their lags; idiosyncratic log-sigmas from the PC residual variances; mu from sample means. PCA via dsyev on the sample covariance — one more LAPACK routine for the wasm shim, which is instance-eight bait and is hereby predicted in writing.

DECISION D3 (recommended): as above.

4. State space cast

States: k*p factor lags in companion form. Z = [Lambda, 0, ...]; H = diag(s2_i) (>0: the univariate filter's diagonal-H requirement is satisfied BECAUSE idiosyncratic errors are iid this release); T = companion of (A_1..A_p); R selects the contemporaneous block; Q = I_k. Initialization: factors are stationary under the evaluation-error rule above, so a1 = 0, P1 by Lyapunov — no diffuse phase. Missing values in any subset of observables at any t are handled by the univariate filter for free; this is the nowcasting payoff and gets its own test.

5. Staged out (explicitly)

Idiosyncratic AR errors (ar()) on the observation equation — moves the idiosyncratic block into the state vector with H = 0; the engine is ready, the parameter map is release two), per-equation exog lists, static factors, constraints language, predict forms beyond smfactor(), factor-augmented anything.

6. Verification protocol

- k=1: statsmodels DynamicFactor(k_factors=1, factor_order=1 and 2) — loglik to ~4 decimals and loadings up to the sign convention, on a simulated panel and on a real macro panel (WEO growth rates of a handful of countries, which also exercises missing edges).
- k=2, identify(triangular): loglik against statsmodels (rotation- invariant) and loadings against a triangular-constrained fit.
- Ragged edge: drop the last 1-4 observations of some series; loglik + smoothed factor against statsmodels on the identical ragged panel.
- tools/stata_check_dfactor.do mirrors the identified cases.

- Tests are tolerance-first from day one (the v1.6.38 lesson, applied prospectively as promised).

7. The constraints subsystem (ruled in by D2)

A general facility, not a dfactor feature. Release scope:

- `constraint [define] # <linear expression> = <linear expression>` where terms are `[eqname]coef`, bare `coef`, and numeric multiples (`2*[y1]f1`). Also `constraint list` and `constraint drop #|_all`. Definitions are TEXT, stored in the workspace, parsed at estimation time against the estimator's parameter names — Stata's semantics (a constraint mentioning parameters the model doesn't have is an error at estimation, not at definition).
- Estimators accept `constraints(numlist)` (dfactor now; sspace when it arrives; others as needed).
- Mechanics: the parsed set becomes $R\theta = r$. A particular solution θ_p (minimum norm, `dgelsd`) and an orthonormal null-space basis N (SVD, `dgesdd`) reparameterize $\theta = \theta_p + N\psi$; the existing BFGS2/multistart/OIM machinery runs UNCHANGED on ψ , and $V\theta = N V_\psi N'$ by the delta method. Constrained coefficients print with 0 SE and "(constrained)" as in Stata.
- This handles every LINEAR constraint exactly, which is the entire Stata constraint language.

DECISION D4 (signed with D2): as above.